17 Django-marcador

Introducción

¿Qué es Django?

El framework web Django es un framework gratuito y de código abierto que puede acelerar el desarrollo de una aplicación web que se crea en el lenguaje de programación Python. En esta web técnica podemos leer un resumen de lo que ofrece Django y su funcionamiento.

Este tutorial nos va a guiar en el desarrollo y despliegue de una app en la que los usuarios podrán registrar sus marcadores (**bookmarks**) de internet y clasificarlos mediante etiquetas (**tags**). Para ello usaremos las siguientes tecnologías y proveedores:

- ➔ Framework web Django.
- ➔ Bases de datos MySQL alojada en Aiven.
- ➔ Github.
- ➔ Servicio de alojamiento web en Render.
- ➔ Cron-job.

Para profundizar en cada tema que se trata en esta guía podemos podemos hacerlo en la documentación de referencia de Django.

Comencemos:

Instalar Django

Para instalar Django usamos el siguiente comando desde el terminal

```
pip install django
```

para ver si se instaló correctamente:

```
Terminal    Local
PS C:\Users\34620\Desktop\marcador> python -m django --version
6.0
PS C:\Users\34620\Desktop\marcador>
```

17 Django-marcador

Un nuevo proyecto de Django

En Django vamos a tener por una lado proyectos, y por otro lado aplicaciones (apps), por ejemplo, un proyecto sería una **tienda virtual**, y luego podemos tener apps tales como **carrito de la compra**, **seguimiento de los pedidos**, **gestión de categorías** o un **Blog anunciando novedades**.

En nuestro ejemplo vamos a tener un proyecto llamado **mysite**, y una app llamada **django-marcador**.

Creamos un directorio:

```
mkdir djangotutorial
```

Puedes crear un nuevo proyecto Django con el siguiente comando:

```
django-admin startproject mysite djangotutorial
```

Esto creará un proyecto llamado mysite dentro del directorio djangotutorial. Veamos lo que startproject creó:

```
djangotutorial/
    manage.py
    mysite/
        __init__.py
        settings.py
        urls.py
        asgi.py
        wsgi.py
```

Estos archivos son:

manage.py Una utilidad de línea de comandos que te permite interactuar con este proyecto de Django de diversas maneras.

mysite/: Un directorio que contiene el paquete de Python de tu proyecto.

mysite/__init__.py: Un archivo vacío que indica a Python que este directorio debe considerarse un paquete.

mysite/settings.py: Configuración de este proyecto de Django. La configuración de Django te explicará todo sobre su funcionamiento.

mysite/urls.py: Las declaraciones de URL para este proyecto de Django; una tabla de contenido del sitio web.

mysite/asgi.py: Un punto de entrada para servidores web compatibles con ASGI (Asynchronous Server Gateway Interface) que permite a los servidores

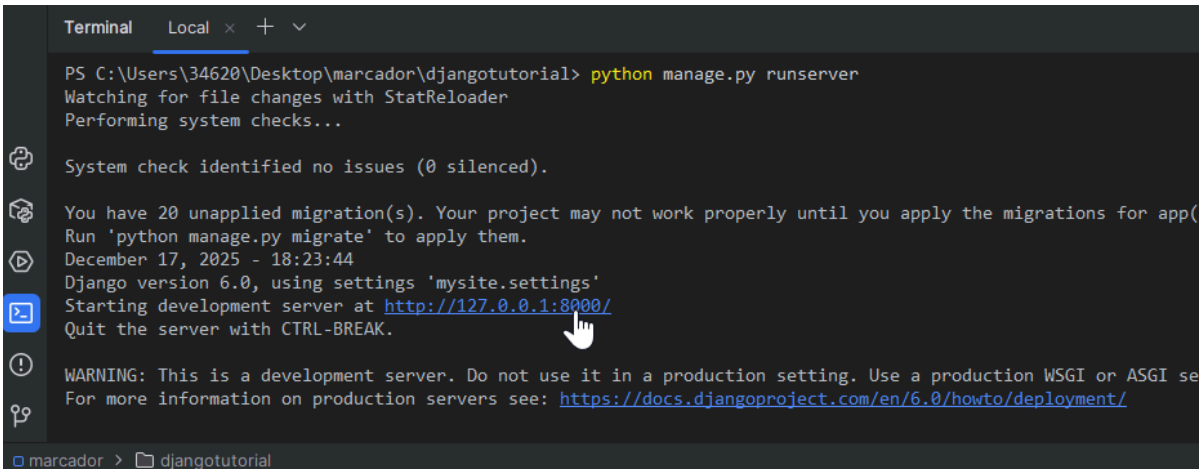
web comunicarse con aplicaciones Python de manera asíncrona. Es el sucesor de WSGI.

mysite/wsgi.py: Un punto de entrada para servidores web compatibles con WSGI (Web Server Gateway Interface) que sirvan nuestro proyecto.

Verifiquemos que tu proyecto de Django funcione. Si aún no lo has hecho, accede al directorio y ejecuta los siguientes comandos:

```
python manage.py runserver
```

Verás la siguiente salida en la línea de comando:



La aplicación django-marcador

Cada aplicación que escribes en Django consta de un paquete de Python que sigue una convención específica. Django incluye una utilidad que genera automáticamente la estructura básica de directorios de una aplicación, para que puedas centrarte en escribir código en lugar de crear directorios.

Proyectos vs Aplicaciones

¿Cuál es la diferencia entre un proyecto y una aplicación? Una aplicación es una aplicación web que realiza alguna función, por ejemplo, un sistema de blogs, una base de datos de registros públicos o una pequeña aplicación de encuestas. Un proyecto es un conjunto de configuraciones y aplicaciones para un sitio web específico. Un proyecto puede contener varias aplicaciones. Una aplicación puede estar en varios proyectos.

Tus aplicaciones pueden estar en cualquier parte de tu ruta de Python. En este tutorial, crearemos nuestra aplicación marcador dentro de la carpeta djangotutorial.

17 Django-marcador

Para crear tu aplicación, asegúrate de estar en el mismo directorio que manage.py y escribe este comando:

```
python manage.py startapp marcador
```

Esto creará un directorio marcador, que se presenta de la siguiente manera:

```
marcador/
    __init__.py
    admin.py
    apps.py
    migrations/
        __init__.py
    models.py
    tests.py
    views.py
```

Esta estructura de directorio albergará la aplicación marcador.

Por último, agrega tu nuevo marcador de aplicación a INSTALLED_APPS en el archivo settings.py:

```
INSTALLED_APPS = (
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'marcador',
)
```

Crear los modelos

Los modelos son el corazón de cualquier aplicación en Django. Básicamente, son la fuente única y definitiva de información sobre tus datos.

En términos simples: un modelo es una clase de Python que Django traduce automáticamente a una tabla de base de datos.

Aquí tienes los conceptos clave para entender cómo funcionan:

1. La estructura básica

Cada atributo de la clase representa un campo (columna) en la base de datos. Django se encarga de todo el SQL por ti.

```
from django.db import models

class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    disponible = models.BooleanField(default=True)
```

2. El flujo de trabajo (Migraciones)

Cuando creas o modificas un modelo, sigues este proceso de dos pasos:

- ➔ makemigrations: Django lee tus modelos y crea un archivo con las instrucciones de los cambios.
- ➔ migrate: Django aplica esos cambios a la base de datos real.

3. Tipos de campos comunes

Django ofrece una gran variedad de campos para validar tus datos desde el inicio:

- ➔ CharField: Para textos cortos (requiere un max_length).
- ➔ TextField: Para textos largos (descripciones, comentarios).
- ➔ IntegerField: Para números enteros.
- ➔ EmailField: Valida automáticamente que el formato sea un correo.

4. Relaciones entre modelos

Esta es la parte más potente. Puedes conectar tablas de tres formas:

1. ForeignKey (Uno a muchos): Un autor tiene muchos libros.
2. OneToOneField (Uno a uno): Un usuario tiene un solo perfil.
3. ManyToManyField (Muchos a muchos): Un estudiante puede estar en muchos cursos, y un curso tiene muchos estudiantes.

Con esta información Django puede generar automáticamente una interfaz a la base de datos que permite el acceso orientado a objetos a ella. Esto se llama Object-Relational Mapping (Mapeo Objeto-Relacional).

- ➔ Podemos leer un análisis comparativo de **Django ORM** con otro potente ORM como es **SQLAlchemy** en esta web técnica.
- ➔ Podemos leer más sobre la creación de modelos en la documentación de Django.

17 Django-marcador

En **django-marcador**, se requieren dos modelos:

1. Bookmark representa los marcadores y tiene campos para URL, título y descripción. Aquí también se almacenan datos sobre quién creó el marcador, cuándo lo creó y si es público. Estos datos se usarán posteriormente para filtrar los marcadores.
2. Tag representa palabras clave que se pueden adjuntar a los marcadores para facilitar su búsqueda.

Bookmark y Tag tienen una relación de muchos a muchos, por lo que Django creará la tabla intermedia automáticamente.

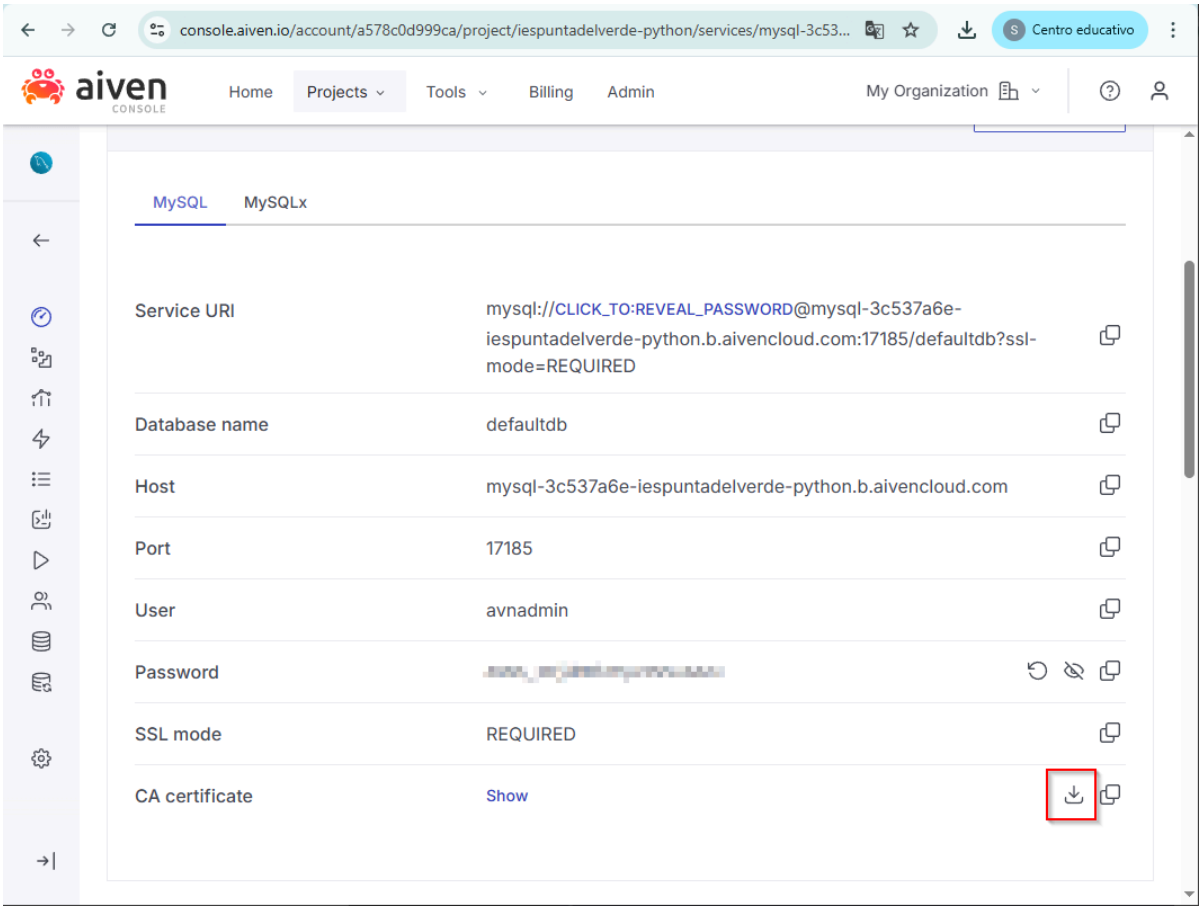
Para que Django reconozca tu clase como un modelo, debe heredar de django.db.models.Model y estar ubicada en models.py en el directorio de tu aplicación (en este ejemplo mysite/marcador/).

```
from django.conf import settings
from django.db import models

class Tag(models.Model):
    name = models.CharField(max_length=50, unique=True)

    def __str__(self):
        return self.name

class Bookmark(models.Model):
    url = models.URLField()
    title = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    is_public = models.BooleanField(default=True)

    # Fechas automáticas (eliminan la necesidad de un método save)
    date_created = models.DateTimeField(auto_now_add=True)
    date_updated = models.DateTimeField(auto_now=True)

    # Relaciones (on_delete es obligatorio)
    owner = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name='bookmarks')
    tags = models.ManyToManyField(Tag, blank=True)

    def __str__(self):
        return f"{self.title} ({self.url})"
```

17 Django-marcador

Database

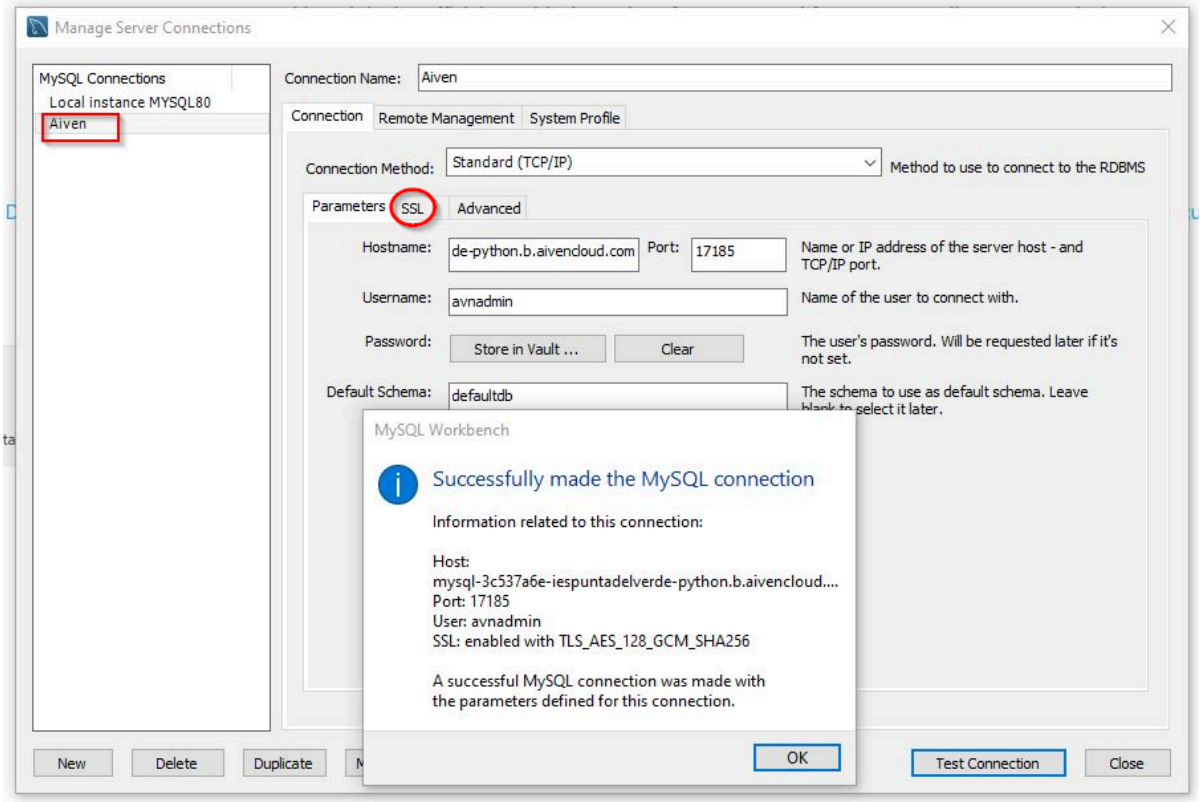
En nuestro ejemplo vamos a usar una base de datos MySQL que está alojada en la plataforma Aiven. En el tema de conexión a bases de datos ya hicimos una conexión a una base de datos alojada en esta plataforma que aprovecharemos para nuestro ejemplo, en mi caso son los siguientes datos:



Descargamos el CA certificate (ca.pem) y lo colocamos en la carpeta principal de nuestro proyecto, junto al manage.py.

Una conexión con Workbench a está base de datos quedaría de la siguiente manera:

17 Django-marcador



OJO! en la pestaña SSL poner como requerido y subir el ca.pem.

en Python tenemos que modificar settings.py:

```
# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'defaultdb',
        'USER': 'avnadmin',
        'PASSWORD': '**************',
        'HOST':
'mysql-3c537a6e-iespuntadelverde-python.b.aivencloud.com',  # ej:
'host-xyz.aiven.io'
        'PORT': '17185',  # ej: '12345'
        'OPTIONS': {
            'ssl': {
                'ca': os.path.join(BASE_DIR, 'ca.pem'),
            },
            'ssl_mode': 'REQUIRED',
        }
    }
}
```

ya tenemos la base de datos lista para hacer una migración.

Database Migration

Las migraciones son la forma que tiene Django de propagar los cambios que realizas en tus modelos (agregar un campo, eliminar un modelo, etc.) en el esquema de tu base de datos.

Nuestra base de datos

```
python manage.py makemigrations marcador

Migrations for 'marcador':
  0001_initial.py:
    - Create model Bookmark
    - Create model Tag
    - Add field tags to bookmark
```

Esto creará el nuevo archivo 0001_initial.py en el directorio migraciones de la aplicación marcador:

```
marcador
|-- __init__.py
|-- admin.py
|-- migrations
|   |-- 0001_initial.py
|   `-- __init__.py
|-- models.py
|-- tests.py
`-- views.py
```

Después de eso, debes aplicar la migración que acabas de crear, así como las migraciones ya existentes para crear el esquema de base de datos utilizando el comando migrate:

```
python manage.py migrate

Operations to perform:
  Synchronize unmigrated apps: staticfiles, messages
  Apply all migrations: admin, contenttypes, marcador, auth, sessions
Synchronizing apps without migrations:
  Creating tables...
    Running deferred SQL...
  Installing custom SQL...
Running migrations:
  Rendering model states... DONE
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
```

```
  Applying admin.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying marcador.0001_initial... OK
  Applying sessions.0001_initial... OK
```

Ahora todas las tablas definidas en las migraciones se han creado en la base de datos.

Admin Site

Configura el Admin Site

Una de las partes más potentes de Django es la interfaz de administración automática. Esta lee los metadatos de tu modelo para proporcionar una interfaz potente y lista para producción que los productores de contenido pueden usar inmediatamente para empezar a añadir contenido al sitio.

Abra el archivo admin.py en el directorio marcador y agregue el siguiente contenido:

```
from django.contrib import admin
from .models import Bookmark, Tag

@admin.register(Tag)
class TagAdmin(admin.ModelAdmin):
    pass  # Registra Tag de forma sencilla

@admin.register(Bookmark)
class BookmarkAdmin(admin.ModelAdmin):
    list_display = ('title', 'owner', 'is_public', 'date_updated')
    list_editable = ('is_public',)
    list_filter = ('is_public', 'owner')
    search_fields = ('url', 'title', 'description')
    readonly_fields = ('date_created', 'date_updated')
```

Crea un superuser

Para crear un nuevo superusuario ejecute el comando createsuperuser y complete los campos que siguen:

```
python manage.py createsuperuser
```

17 Django-marcador

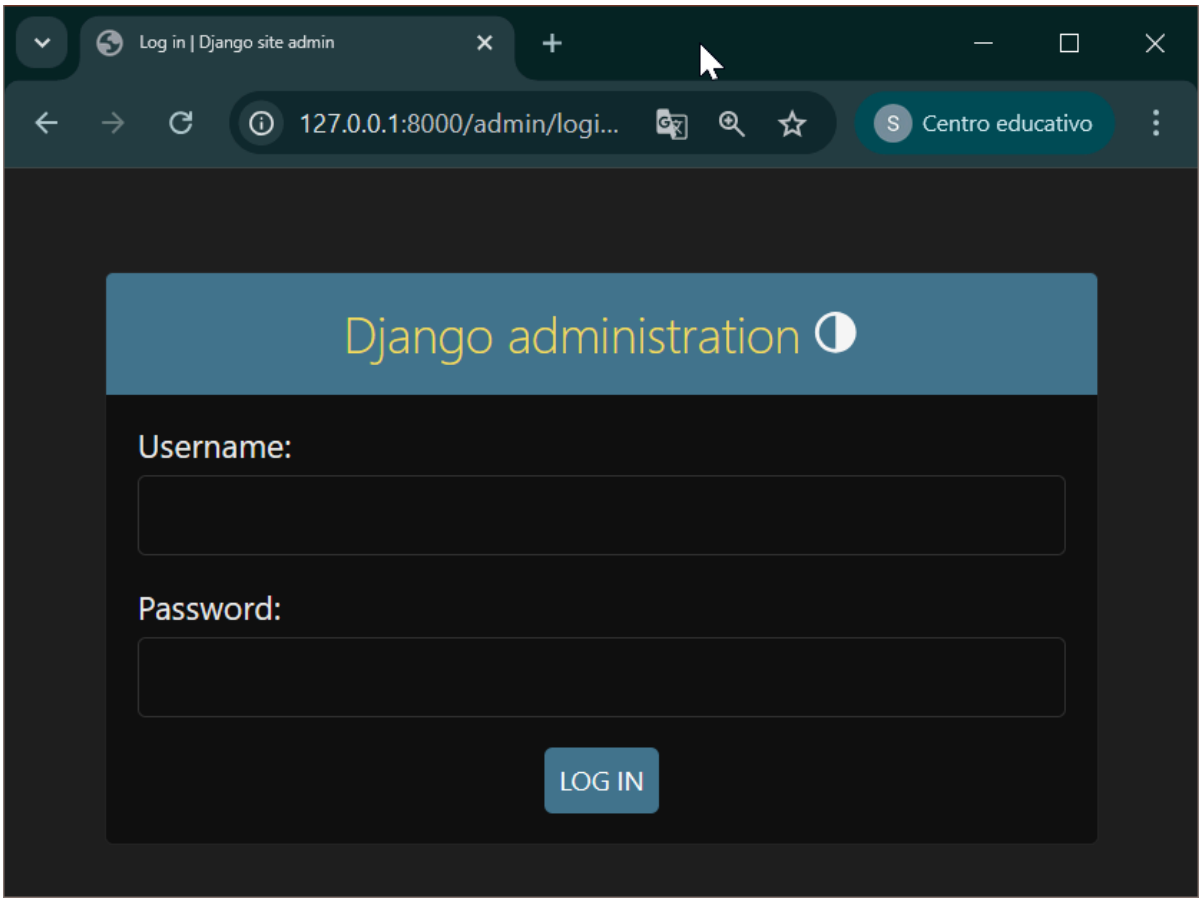
```
Username: admin
Email address:
Password:
Password (again):
Superuser created successfully.
```

En el siguiente paso, podrá iniciar sesión con estos datos de acceso. Abra el sitio de administración e Inicie el servidor de desarrollo:

```
python manage.py runserver
```

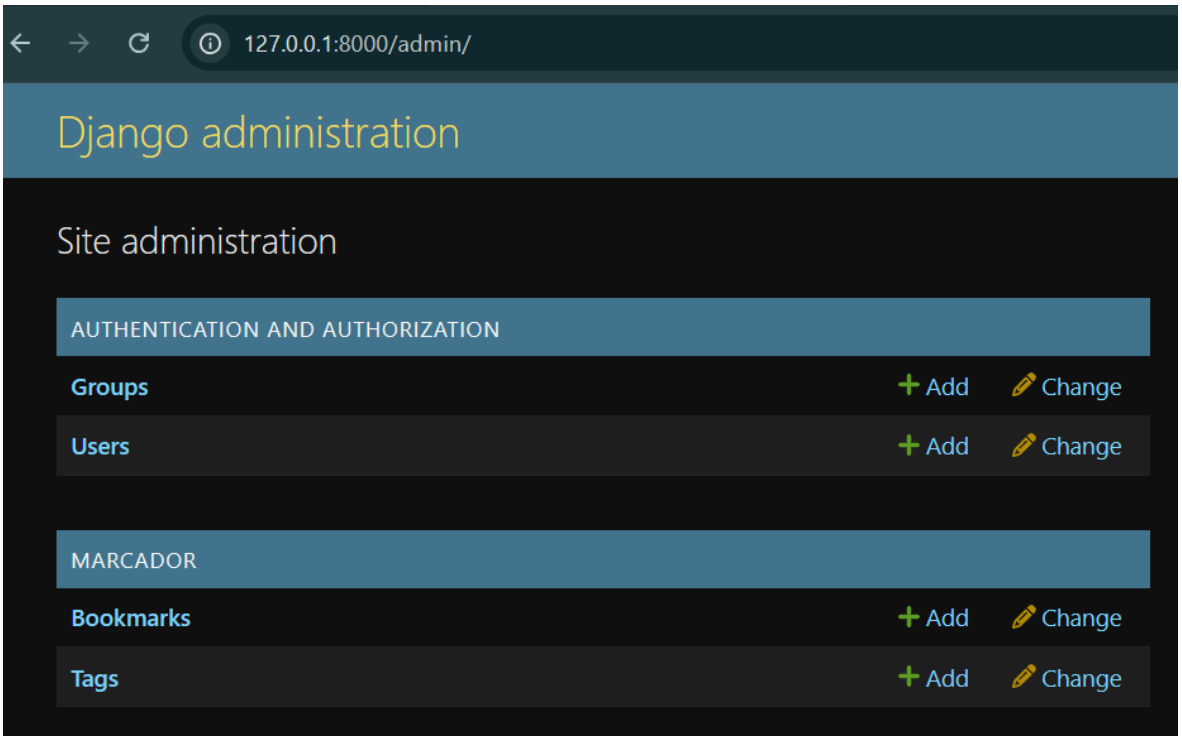
Ahora puedes visitar el sitio de administración en http://127.0.0.1:8000/admin/ e iniciar sesión con el usuario que creaste anteriormente para guardar tus marcadores.

Así se ve el formulario de inicio de sesión:



Después de iniciar sesión, verá la página de administración del sitio:

17 Django-marcador



Introduciendo datos automáticamente

Vamos a introducir algunos datos para que la base de datos se vea realista. Como el modelo de Bookmark requiere un owner y opcionalmente tags, asegúrate de tener al menos un usuario creado en tu base de datos antes de insertar estos datos.

El siguiente script (poblar_bookmarks.py) introduce marcadores, tags y hace algunas asociaciones de forma aleatoria, después de ejecutarlo tendremos marcadores con sus tags asociados.

```
import random
from django.contrib.auth import get_user_model
from marcador.models import Bookmark, Tag

User = get_user_model()

def run():
    print("Iniciando la población de datos...")

    # 1. Obtener o crear un usuario para asignar los marcadores
    user, created = User.objects.get_or_create(
        username='admin',
        defaults={'is_staff': True, 'is_superuser': True}
    )
```

17 Django-marcador

```
    if created:
        user.set_password('admin123')
        user.save()
        print(f"Usuario '{user.username}' creado.")

    # 2. Definir 15 Tags
    nombres_tags = [
        'Programación', 'Python', 'Django', 'Frontend', 'Backend',
        'IA', 'DevOps', 'Cloud', 'Diseño', 'Productividad',
        'Tutoriales', 'Noticias', 'Bases de Datos', 'Seguridad', 'Open
Source'
    ]

    tags_objetos = []
    for nombre in nombres_tags:
        tag, _ = Tag.objects.get_or_create(name=nombre)
        tags_objetos.append(tag)
    print(f"Se han verificado/creado {len(tags_objetos)} etiquetas.")

    # 3. Definir 30 Marcadores (Título y URL)
    datos_bookmarks = [
        ("Google", "https://www.google.com"), ("GitHub",
"https://github.com"),
        ("Stack Overflow", "https://stackoverflow.com"), ("Django
Project", "https://www.djangoproject.com"),
        ("Python Official", "https://www.python.org"), ("YouTube",
"https://www.youtube.com"),
        ("Reddit", "https://www.reddit.com"), ("Twitter / X",
"https://twitter.com"),
        ("LinkedIn", "https://www.linkedin.com"), ("MDN Web Docs",
"https://developer.mozilla.org"),
        ("ChatGPT", "https://chat.openai.com"), ("AWS",
"https://aws.amazon.com"),
        ("Vercel", "https://vercel.com"), ("Trello",
"https://trello.com"),
        ("Notion", "https://www.notion.so"), ("Medium",
"https://medium.com"),
        ("Dev.to", "https://dev.to"), ("Figma",
"https://www.figma.com"),
        ("Canva", "https://www.canva.com"), ("Spotify",
"https://www.spotify.com"),
        ("DigitalOcean", "https://www.digitalocean.com"), ("Heroku",
"https://www.heroku.com"),
        ("GitLab", "https://about.gitlab.com"), ("Bitbucket",
"https://bitbucket.org"),
```

```
        ("Laravel", "https://laravel.com"), ("Vue.js",
"https://vuejs.org"),
        ("React", "https://react.dev"), ("Angular",
"https://angular.io"),
        ("Docker", "https://www.docker.com"), ("Kubernetes",
"https://kubernetes.io")
    ]

    # 4. Crear los marcadores y asignar tags aleatorios
    for titulo, url in datos_bookmarks:
        # Usamos update_or_create para evitar duplicados si corres el
script varias veces
        bookmark, created = Bookmark.objects.update_or_create(
            url=url,
            owner=user,
            defaults={
                'title': titulo,
                'description': f"Este es un recurso excelente sobre
{titulo}. Muy útil para el desarrollo diario.",
                'is_public': random.choice([True, True, True, False]),
# Más probabilidad de ser público
            }
        )

        # Obtiene y asigna una lista con entre 1 y 3 tags aleatorios del
pool creado arriba
        tags_aleatorios = random.sample(tags_objetos,
k=random.randint(1, 3))
        bookmark.tags.set(tags_aleatorios)

    print(f"¡Hecho! Se han procesado {len(datos_bookmarks)} marcadores
correctamente.")

if __name__ == "__main__":
    run()
```

Para ejecutar el Script podemos hacerlos desde la shell de Python con los comandos:

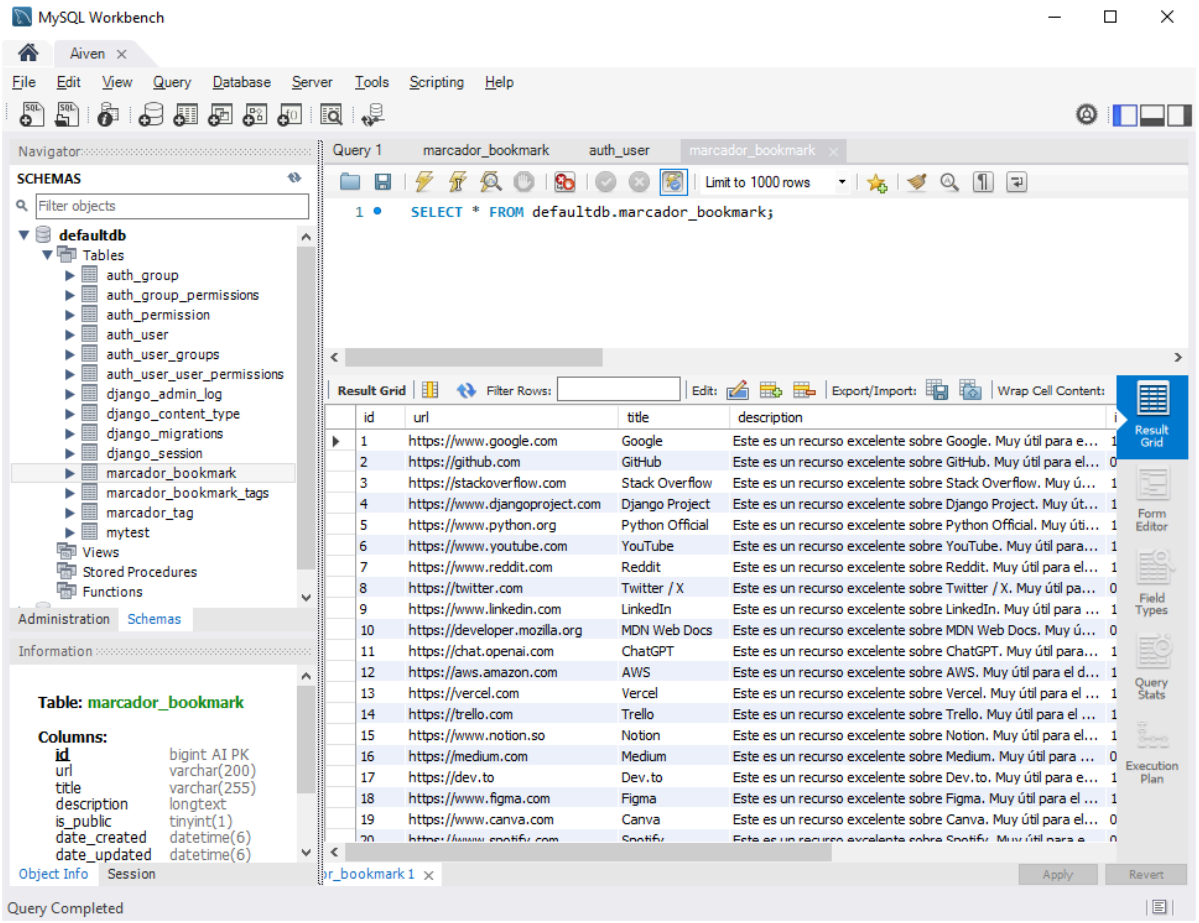
```
>>> exec(open('poblar_bookmarks.py', encoding='utf-8').read())
>>> run()
```

17 Django-marcador

tal como se ve en la imagen:

```
(.venv) PS C:\Users\Salva\Desktop\marcador\djangotutorial> python manage.py shell
14 objects imported automatically (use -v 2 for details).

Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD6
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> exec(open('poblar_bookmarks.py', encoding='utf-8').read())
>>> run()
Iniciando la población de datos...
Se han verificado/creado 15 etiquetas.
¡Hecho! Se han procesado 30 marcadores correctamente.
>>>
```

ya tenemos nuestra base de datos con algunos datos:



URLs, vistas y plantillas

Vamos a darle vida a estos datos. En Django moderno, lo más rápido y limpio es usar una **Class-Based View (CBV)**. Sigue estos tres pasos:

17 Django-marcador

1. La Vista (views.py)

Utilizaremos ListView, que ya trae toda la lógica para consultar la base de datos.

```
from django.views.generic import ListView
from .models import Bookmark

class BookmarkListView(ListView):
    model = Bookmark
    template_name = 'bookmarks_list.html'
    context_object_name = 'bookmarks'

    # Solo mostramos los que marcamos como públicos
    def get_queryset(self):
        return Bookmark.objects.filter(is_public=True).order_by('-date_created')
```

2. La URL (urls.py)

Debes conectar la vista con una dirección web.

```
from django.urls import path
from .views import BookmarkListView

urlpatterns = [
    path('bookmarks/', BookmarkListView.as_view(),
name='bookmark-list'),
]
```

3. El Template (templates/bookmarks_list.html)

Crea un archivo HTML sencillo para mostrar la lista. Django buscará este archivo automáticamente.

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Mis Marcadores</title>
    <style>
        body { font-family: sans-serif; margin: 40px; line-height: 1.6; }
        .bookmark { border-bottom: 1px solid #eee; padding: 10px 0; }
        .tag { background: #e0e0e0; padding: 2px 6px; border-radius: 4px;
font-size: 0.8em; }
    </style>
</head>
```

17 Django-marcador

```
<body>
    <h1>Mis Marcadores Públicos</h1>
    {% for b in bookmarks %}
        <div class="bookmark">
            <a href="{{ b.url }}" target="_blank"><strong>{{ b.title
}}</strong></a>
            <p>{{ b.description }}</p>
            <div>
                {% for tag in b.tags.all %}
                    <span class="tag">{{ tag.name }}</span>
                {% endfor %}
            </div>
        </div>
    {% empty %}
        <p>No hay marcadores disponibles.</p>
    {% endfor %}
</body>
</html>
```

¿Qué acabamos de construir?

1. **Lógica Eficiente**: Django solo trae de la base de datos lo que pedimos en el get_queryset.
2. **Seguridad**: Al usar el motor de plantillas de Django, tus enlaces y textos están protegidos contra ataques comunes como XSS.
3. **Relaciones**: El template recorre automáticamente la relación ManyToMany de las etiquetas (b.tags.all).

Tenemos que verificar los siguiente, verifica el urls.py del Proyecto:

Un proyecto Django suele tener dos archivos urls.py. Asegúrate de que el archivo principal (el que está en la carpeta junto a settings.py) esté "llamando" a las URLs de tu aplicación.

Debería verse así:

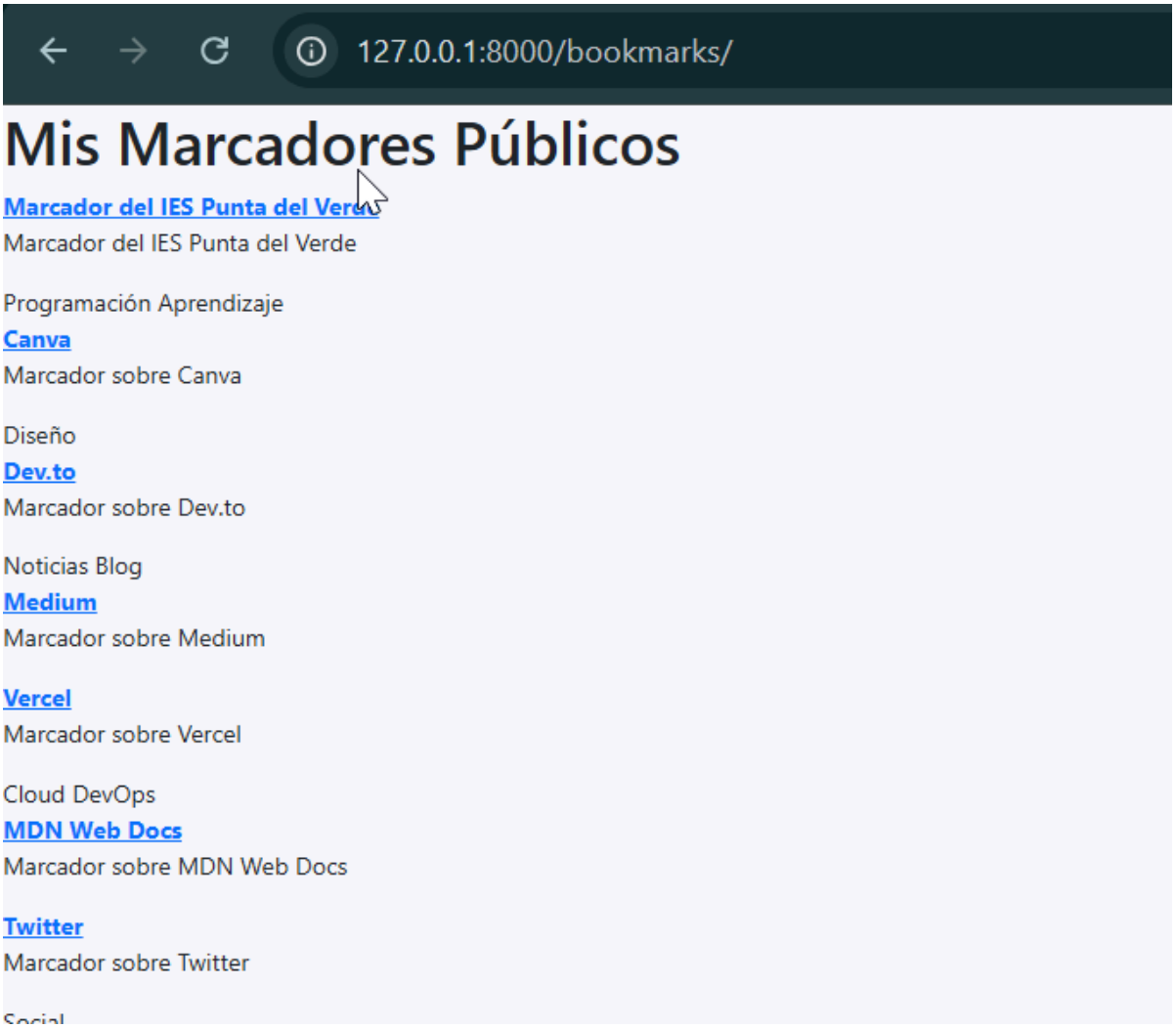
```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('marcador.urls')), # Asegúrate de que esta línea
existe
]
```

Verifica el urls.py de la app:

```
from django.urls import path
from .views import BookmarkListView
```

```
urlpatterns = [
    path('bookmarks/', BookmarkListView.as_view(),
name='bookmark-list'),
]
```

Debería verse así:



4. Aplicar estilos

Vamos a aplicar estilos con CSS, y lo haremos de forma que quede bien estructurado. Aplicamos los siguientes estilos:

- ➔ Grid System: Usamos container y row para centrar el contenido y que sea responsivo (se ve bien en móviles).
- ➔ Cards y Sombras: El contenedor blanco con sombra (card shadow-sm) da profundidad y limpieza.

- ➔ Table Hover: Las filas de la tabla se iluminan al pasar el ratón, mejorando la UX.
- ➔ Badges: Las etiquetas de Tag ahora aparecen como pequeñas cápsulas de color.

Procedamos:

1. Crear la carpeta de estáticos

   Dentro de la aplicación (donde está models.py), crea esta estructura de carpetas: marcador/static/css/style.css

   Nota importante: Django busca automáticamente dentro de las carpetas llamadas static. Es buena práctica crear una subcarpeta css dentro para mantener el orden.

2. El archivo CSS (style.css)

   Corta los estilos que teníamos en el HTML y pégalos aquí. He añadido un par de detalles para que se vea mejor:

CSS

```
/* marcador/static/css/style.css */
body {
    background-color: #f8f9fa;
}

.navbar-brand {
    font-weight: bold;
    letter-spacing: 1px;
}

.card {
    border: none;
    border-radius: 15px;
}

.table thead {
    background-color: #e9ecef;
}

.badge {
    font-weight: 500;
    margin-right: 3px;
}

.btn-visit {
    transition: all 0.3s;
}

.btn-visit:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);
```

```
}
```

3. Vincularlo en el Template (bookmarks_list.html)

Para que Django cargue el archivo, necesitas usar la etiqueta {% load static %} al principio del HTML.

HTML

```
{% load static %} <!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Mis Marcadores</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.mi
n.css" rel="stylesheet">
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body class="bg-light">
    ```
```

4. ¿Dónde colocar cada cosa?

Aquí tienes el mapa visual del proyecto:

```
djangotutorial/
├── manage.py
├── mysite/ (configuración global)
└── marcador/ (tu aplicación)
    ├── static/
    │   └── css/
    │       └── style.css  <-- AQUÍ VA EL CSS
    ├── templates/
    │   └── bookmarks_list.html <-- AQUÍ VA EL HTML
    ├── models.py
    ├── views.py
    └── urls.py
```

Un último detalle: STATIC_URL

Asegúrate de que en tu archivo settings.py exista esta línea (normalmente ya viene por defecto):

Python

```
STATIC_URL = 'static/'
```
```

17 Django-marcador

Formularios

Una de las funciones más potentes de Django son los ModelForms. Básicamente, Django lee tu modelo y construye el formulario HTML por ti, incluyendo la validación de datos. Sigue estos pasos para implementarlo:

1. Crear el archivo forms.py

Crea este archivo en la carpeta de tu aplicación para definir qué campos quieres mostrar.

Python

```
from django import forms
from .models import Bookmark

class BookmarkForm(forms.ModelForm):
    class Meta:
        model = Bookmark
        # Elegimos qué campos queremos que el usuario llene
        fields = ['url', 'title', 'description', 'is_public', 'tags']
        # Añadimos clases de Bootstrap para que se vea bien
        widgets = {
            'url': forms.URLInput(attrs={'class': 'form-control',
'placeholder': 'https://...'}),
            'title': forms.TextInput(attrs={'class': 'form-control'}),
            'description': forms.Textarea(attrs={'class':
'form-control', 'rows': 3}),
            'is_public': forms.CheckboxInput(attrs={'class':
'form-check-input'}),
            'tags': forms.SelectMultiple(attrs={'class':
'form-select'}),
        }
```

2. Crear la Vista (views.py)

Usaremos una CreateView, que maneja automáticamente el guardado en la base de datos.

Python

```
from django.views.generic import CreateView
from django.urls import reverse_lazy
from .models import Bookmark
from .forms import BookmarkForm

class BookmarkCreateView(CreateView):
```

```
    model = Bookmark
    form_class = BookmarkForm
    template_name = 'bookmark_form.html'
    success_url = reverse_lazy('bookmark-list') # Al terminar, vuelve a
la lista

    def form_valid(self, form):
        # Asignamos automáticamente el usuario logueado como dueño
        form.instance.owner = self.request.user
        return super().form_valid(form)
```

3. Registrar la URL (urls.py)

Añade la ruta para acceder al formulario:

Python

```
path('nuevo/', BookmarkCreateView.as_view(), name='bookmark-create'),
```

4. El Template (templates/bookmark_form.html)

Crea este archivo para mostrar el formulario con el estilo de Bootstrap que ya veníamos usando.

HTML y BOOTSTRAP

```
{% load static %}
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Añadir Marcador</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body class="bg-light">

    <div class="container py-5">
        <div class="row justify-content-center">
            <div class="col-md-6 col-lg-5">

                <div class="card shadow-sm card-form">
                    <div class="text-center mb-4">
                        <h2 class="h3 text-dark">Nuevo Marcador</h2>
                        <p class="text-muted small">Guarda tus enlaces
```

```
favoritos</p>
                    </div>

                    <form method="post">
                        {% csrf_token %}

                        {% for field in form %}
                            <div class="mb-3">
                                <label class="form-label">{{ field.label
}}</label>
                                {{ field }}
                                {% if field.help_text %}
                                    <div class="form-text">{{ field.help_text
}}</div>
                                {% endif %}
                                {% for error in field.errors %}
                                    <div class="text-danger small">{{ error
}}</div>
                                {% endfor %}
                            </div>
                        {% endfor %}

                        <div class="d-grid gap-2 mt-4">
                            <button type="submit" class="btn btn-primary
btn-visit">
                                Guardar Marcador
                            </button>
                            <a href="{% url 'bookmark-list' %}" class="btn
btn-outline-secondary">
                                Volver a la lista
                            </a>
                        </div>
                    </form>
                </div>

            </div>
        </div>
    </div>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.j
s"></script>
</body>
</html>
```

- ➔ {% csrf_token %}: Es una etiqueta de seguridad vital en Django que evita ataques externos al enviar formularios.
- ➔ form.as_p: Renderiza cada campo del formulario dentro de etiquetas <p>. (Si quieres más control, puedes renderizar campo por campo).
- ➔ form_valid: Como en tu modelo el campo owner es obligatorio, este método toma al usuario que está usando la web y lo asigna al marcador antes de guardarlo.

y añadimos estas lineas al final del archivo CSS:

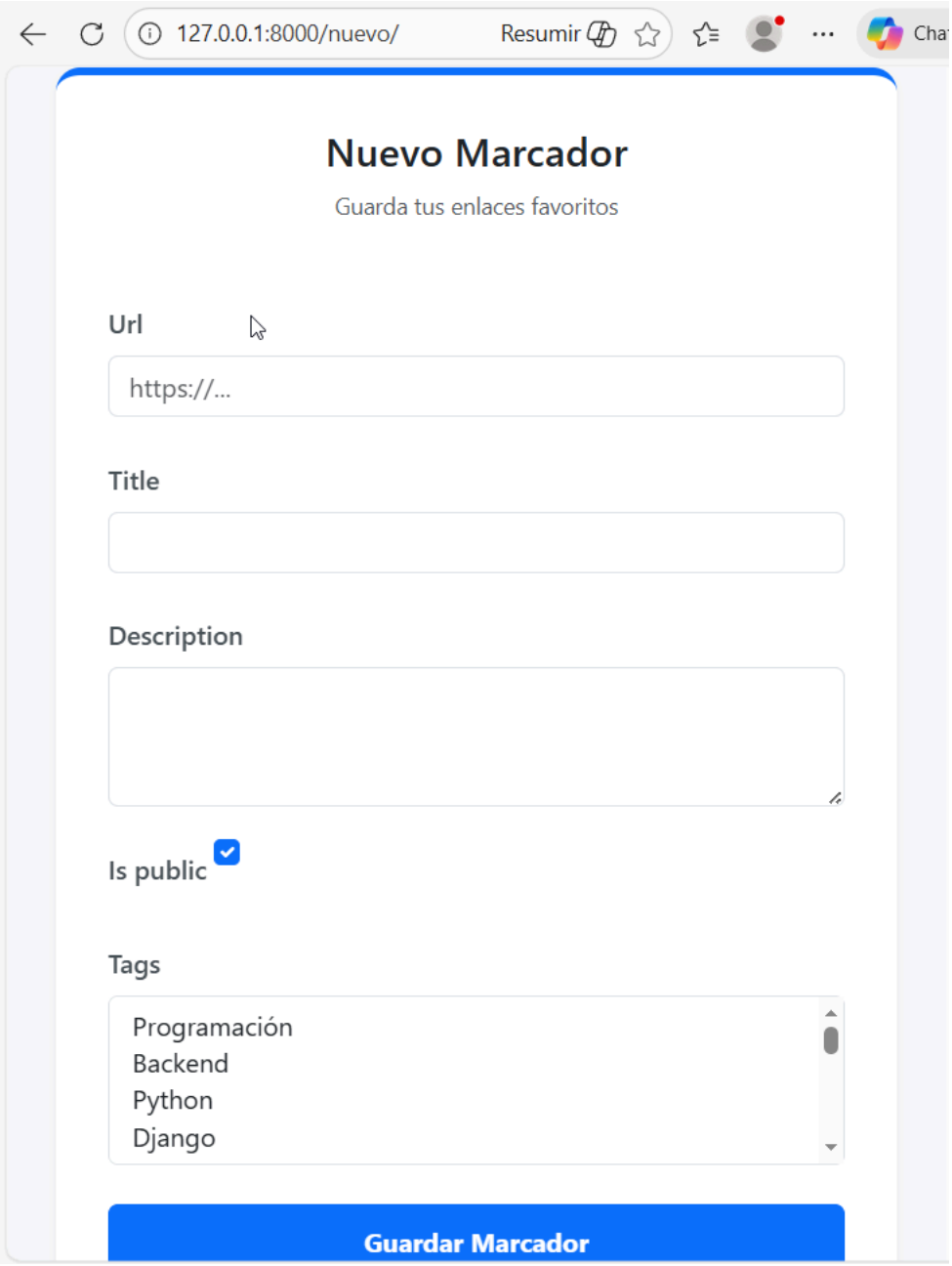
CSS

```
/* Estilos extra para el formulario */
.card-form {
    border-top: 5px solid #0d6efd; /* Una línea azul arriba */
    padding: 2rem;
}

.form-label {
    font-weight: 600;
    color: #495057;
    margin-top: 10px;
}

.btn-primary {
    padding: 10px;
    font-weight: bold;
}
```

¡Ojo! Para que esto no te de error, asegúrate de estar logueado en el admin antes de entrar a /nuevo/, ya que el script necesita un usuario para asignar el campo owner.

Debería verse así:

17 Django-marcador



Filtros

Vamos a añadir una **barra de filtros** en la parte superior y convertiremos las etiquetas (Tags) en **enlaces clicables**. Esta es una de las funcionalidades más útiles porque permite organizar cientos de marcadores con un solo clic. Para que esto funcione, necesitamos modificar la **Vista** (para que sepa filtrar) y el **Template** (para que el usuario tenga dónde hacer clic).

Así, si ves un marcador con la etiqueta "Python", puedes hacer clic directamente en ella para filtrar.

17 Django-marcador

1. Modifica la Vista (views.py)

Vamos a actualizar tu BookmarkListView. El truco está en capturar lo que viene en la URL después del signo ?.

```
class BookmarkListView(LoginRequiredMixin, ListView):
    model = Bookmark
    template_name = 'bookmarks_list.html'
    context_object_name = 'bookmarks'

    def get_queryset(self):
        # FILTRO CLAVE: Solo marcadores que pertenecen al usuario
logueado
        queryset =
Bookmark.objects.filter(owner=self.request.user).order_by('-date_created
')

        # Filtrado por etiquetas
        tag_name = self.request.GET.get('tag')
        if tag_name:
            queryset = queryset.filter(tags__name__iexact=tag_name)

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        # Opcional: Mostrar solo etiquetas que el usuario realmente está
usando
        context['all_tags'] =
Tag.objects.filter(bookmark__owner=self.request.user).distinct()
        context['tag_actual'] = self.request.GET.get('tag')
        return context
```

2. Actualiza el Template (bookmarks_list.html)

bookmarks_list.html (Actualizado para Filtros)

```
{% load static %}
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Mis Marcadores</title>
```

17 Django-marcador

```
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.mi
n.css" rel="stylesheet">
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body class="bg-light">
    <div class="container mt-5">
        <h1>Mis Marcadores Públicos</h1>

        <div class="my-4 p-3 bg-white shadow-sm rounded">
            <span class="text-muted me-2">Filtrar por etiqueta:</span>
            <a href="{% url 'bookmark-list' %}" class="badge
bg-secondary text-decoration-none">Todos</a>

            {% for tag in all_tags %}
                <a href="?tag={{ tag.name }}"
                   class="badge {% if tag_actual == tag.name
%}bg-primary{% else %}bg-info text-dark{% endif %}
text-decoration-none">
                    {{ tag.name }}
                </a>
            {% endfor %}
        </div>

        <hr>

        {% for b in bookmarks %}
        <div class="bookmark mb-4 p-3 border-bottom">
            <a href="{{ b.url }}" target="_blank"><strong>{{ b.title
}}</strong></a>
            <p class="mb-1 text-secondary">{{ b.description }}</p>
            <div>
                {% for tag in b.tags.all %}
                <a href="?tag={{ tag.name }}" class="tag badge bg-info
text-dark text-decoration-none">
                    {{ tag.name }}
                </a>
                {% endfor %}
            </div>
        </div>
        {% empty %}
        <p class="alert alert-warning">No hay marcadores disponibles
para esta selección.</p>
        {% endfor %}
```

```
        <div class="mt-4">
            <a href="{% url 'bookmark-create' %}" class="btn
btn-success">+ Añadir nuevo marcador</a>
        </div>
    </div>
</body>
</html>
```

¿Qué cambios hemos hecho?

1. **Barra Superior**: Hemos añadido un contenedor blanco con all_tags. Cada etiqueta apunta a ?tag={{ tag.name }}.
2. **Lógica de Color**: Si la etiqueta en la que estás es la "activa" (tag_actual), se pone de color azul oscuro (bg-primary) para que sepas dónde estás.
3. **Etiquetas Clicables**: Dentro de cada marcador, las etiquetas ya no son solo texto, ahora son enlaces <a> para que navegar sea mucho más rápido.
4. **Contenedor Bootstrap**: Hemos añadido la clase container para que el texto no se pegue a los bordes de la pantalla y se vea más ordenado.

Recuerda: Para que esto funcione, asegúrate de que tu función get_context_data en views.py esté enviando all_tags y tag_actual, como vimos en el paso anterior.

Tenemos que hacer una modificación en views.py:

Ve a tu archivo views.py y asegúrate de que la línea donde importas tus modelos incluya tanto a Bookmark como a Tag. Debería verse así:

Python

```python
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView, CreateView
from django.urls import reverse_lazy
from .models import Bookmark, Tag # <--- ASEGÚRATE DE QUE 'Tag' ESTÉ AQUÍ
from .forms import BookmarkForm
```

Revisa también: get_context_data

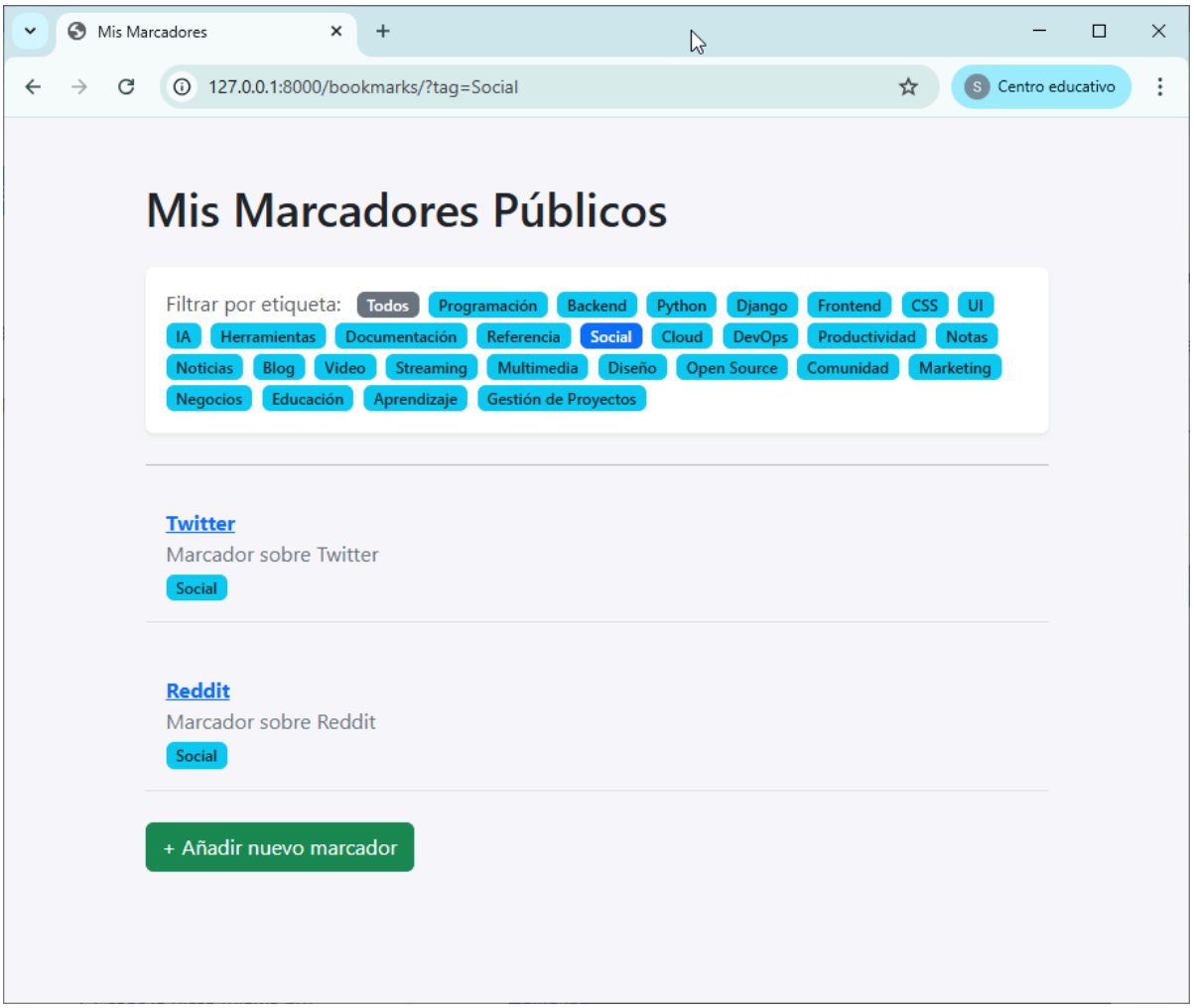
Para que el template no te dé error, asegúrate de que la vista tenga este bloque:

Python

```python
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
```

```
        context['all_tags'] = Tag.objects.all() # Aquí usamos Tag
        context['tag_actual'] = self.request.GET.get('tag')
        return context
```

Debería de verse ahora así:



El proyecto completo está disponible en:

https://github.com/salvadormartos/marcador.git

Despliegue

Una vez que tu sitio esté terminado (o lo suficientemente terminado como para iniciar una prueba pública) vas a necesitar alojarla en un lugar más público y accesible que tu computadora personal.

Hasta ahora has estado trabajando en un entorno de desarrollo, usando Django development web server para compartir tu sitio con el navegador/red local, y corriendo tu sitio web con configuraciones (inseguras) de

desarrollo que exponen la depuración y otra información privada. Antes de que puedas alojar tu sitio web externamente, lo primero que tendrás que hacer es:

- ➔ Introducir algunos cambios en las configuraciones del proyecto.
- ➔ Elegir un entorno para alojar la aplicación Django.
- ➔ Elegir un entorno para alojar cualquier archivo estático.
- ➔ Configurar una infraestructura a nivel producción para servir tu sitio web.

Eligiendo un proveedor de hosting

"Django Friendly Hosts" se refiere a proveedores y plataformas de alojamiento web ideales para implementar aplicaciones Django, desde servicios gestionados para principiantes como PythonAnywhere, Render y Railway hasta potentes opciones de IaaS/PaaS como DigitalOcean, AWS y Google Cloud Platform (GCP) para mayor control y escalabilidad, con recursos comunitarios como Djangofriendly.com que ofrecen reseñas y recomendaciones. Los factores clave son la facilidad de configuración, el coste, el rendimiento y funciones específicas como la compatibilidad con bases de datos; las opciones suelen depender de tus habilidades técnicas y la escala del proyecto.

Plataformas gestionadas y fáciles de usar para principiantes:

- ➔ PythonAnywhere: Excelente punto de partida para principiantes con una configuración sencilla, pero requiere actualización para dominios personalizados/SSH.
- ➔ Render: Una moderna plataforma como servicio (PaaS) ideal para prototipos y producción, con precios competitivos.
- ➔ Railway y Koyeb: Modelos de facturación basados en el uso, ideales para aplicaciones y prototipos más pequeños.
- ➔ Heroku: Popular entre startups, pero ya no ofrece dynos web gratuitos, ya que los costes aumentan a medida que aumenta la escala.

Vamos a desplegar nuestra aplicación en Render.

Desplegando nuestra App en Render

Tendremos que realizar las siguientes tareas,

Preparación de tu Proyecto Django:

1. **Variables** de Entorno: Usa python-decouple para manejar SECRET_KEY, DEBUG, DATABASE_URL, etc., desde variables de entorno.
2. **Base de Datos**: Configura settings.py para conectarse a MySQL usando dj-database-url o similar, leyendo la URL de la base de datos de una variable de entorno (ej: DATABASE_URL).

3. **Servidor WSGI**: Añade unicorn como servidor WSGI: pip install gunicorn (o uvicorn si es para ASGI) y configúralo en tu settings.py.
4. **Archivos Estáticos**: Configura el manejo de archivos estáticos para producción (ej. whitenoise).
5. **.gitignore**: Asegúrate de ignorar *.sqlite3, archivos de caché y variables de entorno locales.

Configuración de Render:

1. **Conecta GitHub**: Conecta tu repositorio de GitHub a Render.
2. **Crea un Servicio de Base de Datos (MySQL)**: Ya tenemos uno creado en Aiven. Configura las variables (nombre, usuario, contraseña).
3. **Crea un Servicio Web (Django)**:
    a. Selecciona tu repositorio de Django.
    b. Configura el Build Command (ej: pip install -r requirements.txt) y Start Command (ej: gunicorn marcador.wsgi:application).
    c. Añade variables de entorno para tu conexión a la base de datos (la URL que Render te dará para el servicio MySQL) y otras variables de Django.
    d. Habilita el despliegue automático (Auto Deploy) en la configuración del servicio para que cada push a tu repo se despliegue.

Por lo que haremos lo siguiente:

- ➔ pip install gunicorn
    - ◆ comando para instalar Gunicorn, un servidor HTTP WSGI (Web Server Gateway Interface) para aplicaciones Python, permitiendo que tu aplicación (como Flask o Django) se ejecute en producción de manera eficiente, gestionando múltiples solicitudes simultáneamente.
- ➔ pip install python-decouple
    - ◆ comando para instalar una librería de Python llamada `python-decouple`, que sirve para separar la configuración de tu aplicación (variables de entorno, claves secretas) del código fuente, manteniéndolo más limpio y seguro, cargando ajustes desde archivos .env o el entorno del sistema sin modificar el código.
- ➔ pip install whitenoise
    - ◆ comando para instalar la biblioteca Whitenoise, una herramienta que permite a aplicaciones web Python (especialmente Django) servir sus propios archivos estáticos (CSS, JavaScript, imágenes) de forma sencilla y eficiente, eliminando la necesidad de servidores externos, ideal para despliegues en plataformas PaaS. Hay que hacer cambios en settings.py:

```
MIDDLEWARE = [
```

```
    # ...
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
    # ...
]
```

- ➔ python -m pip freeze > requirements.txt
  - ◆ comando que registra la lista de paquetes actual del entorno en el archivo requirements.txt.
- ➔ python manage.py collectstatic --no-input
  - ◆ comando de Django que copia todos los archivos estáticos (CSS, JavaScript, imágenes) de tus aplicaciones a una única carpeta (definida por STATIC_ROOT) para su despliegue en producción, y el argumento --no-input le indica que lo haga sin pedir confirmación, asumiendo que quieres sobrescribir los archivos existentes.
- ➔ crear archivo Procfile (sin extensión) en la raíz del proyecto
  - ◆ El Procfile es un archivo de texto en la raíz de tu aplicación (especialmente en plataformas PaaS como Heroku) que define los comandos para iniciar procesos; web: gunicorn mysite.wsgi --log-file - le dice a la plataforma que inicie un servidor web usando Gunicorn, un servidor WSGI, para servir tu aplicación de Django/Flask definida en mysite.wsgi, enviando los logs a la salida estándar (stdout/stderr) en lugar de un archivo.

```
web: gunicorn mysite.wsgi --log-file -
```

- ➔ crear archivo .env en la raíz del proyecto
  - ◆ Un archivo .env (de "environment") es un archivo de texto plano que almacena variables de configuración sensibles, como contraseñas, claves API o URLs de bases de datos, separándolas del código fuente para mejorar la seguridad y la flexibilidad entre entornos (desarrollo, producción). Se usa comúnmente en desarrollo web (Node.js, Python, etc.) para evitar incluir información confidencial en repositorios públicos y se carga en la aplicación mediante librerías como decouple.

```
DEBUG=false
DB_ENGINE=django.db.backends.mysql
DB_NAME=defaultdb
DB_USER=avnadmin
DB_PASSWORD=your_db_password
DB_HOST=mysql-3c537a6e-iespuntadelverde-python.b.aivencloud.com
DB_PORT=17185
ALLOWED_HOSTS=.onrender.com,localhost,127.0.0.1
SECRET_KEY=your_secret_key
```

17 Django-marcador

Nuestro archivo settings.py quedaría así:

```
import os
from pathlib import Path
from decouple import config, Csv

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = config('SECRET_KEY')

# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'marcador',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',  # <-- whitenoise
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'mysite.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

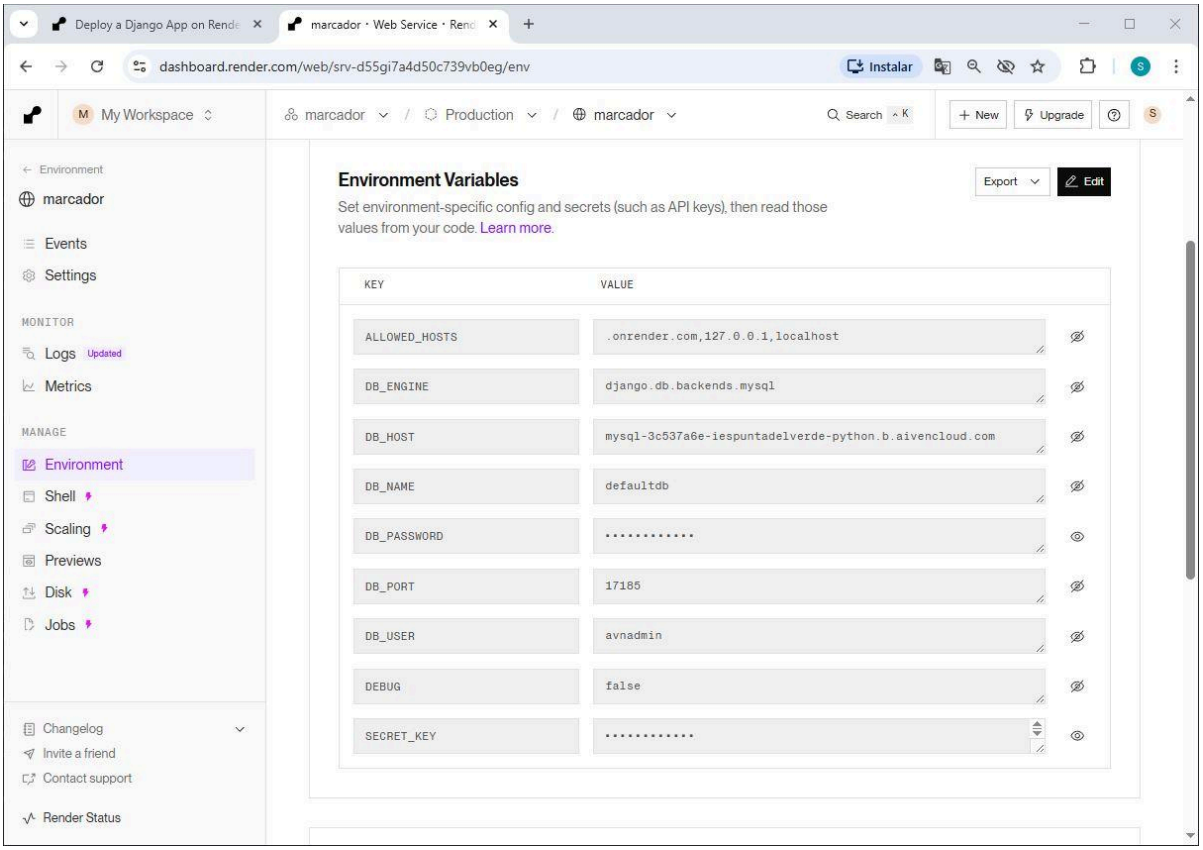
17 Django-marcador

```
WSGI_APPLICATION = 'mysite.wsgi.application'

# Database
DATABASES = {
    'default': {
        'ENGINE': config("DB_ENGINE"),
        'NAME': config("DB_NAME"),
        'USER': config("DB_USER"),
        'PASSWORD': config("DB_PASSWORD"),
        'HOST': config("DB_HOST"),
        'PORT': config("DB_PORT"),
        'OPTIONS': {
            'ssl': {
                'ca': os.path.join(BASE_DIR, 'ca.pem'),
            },
            'ssl_mode': 'REQUIRED',
        }
    }
}

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = config("DEBUG", default=False, cast=bool)
ALLOWED_HOSTS = config('ALLOWED_HOSTS', cast=Csv())

# Password validation
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
```

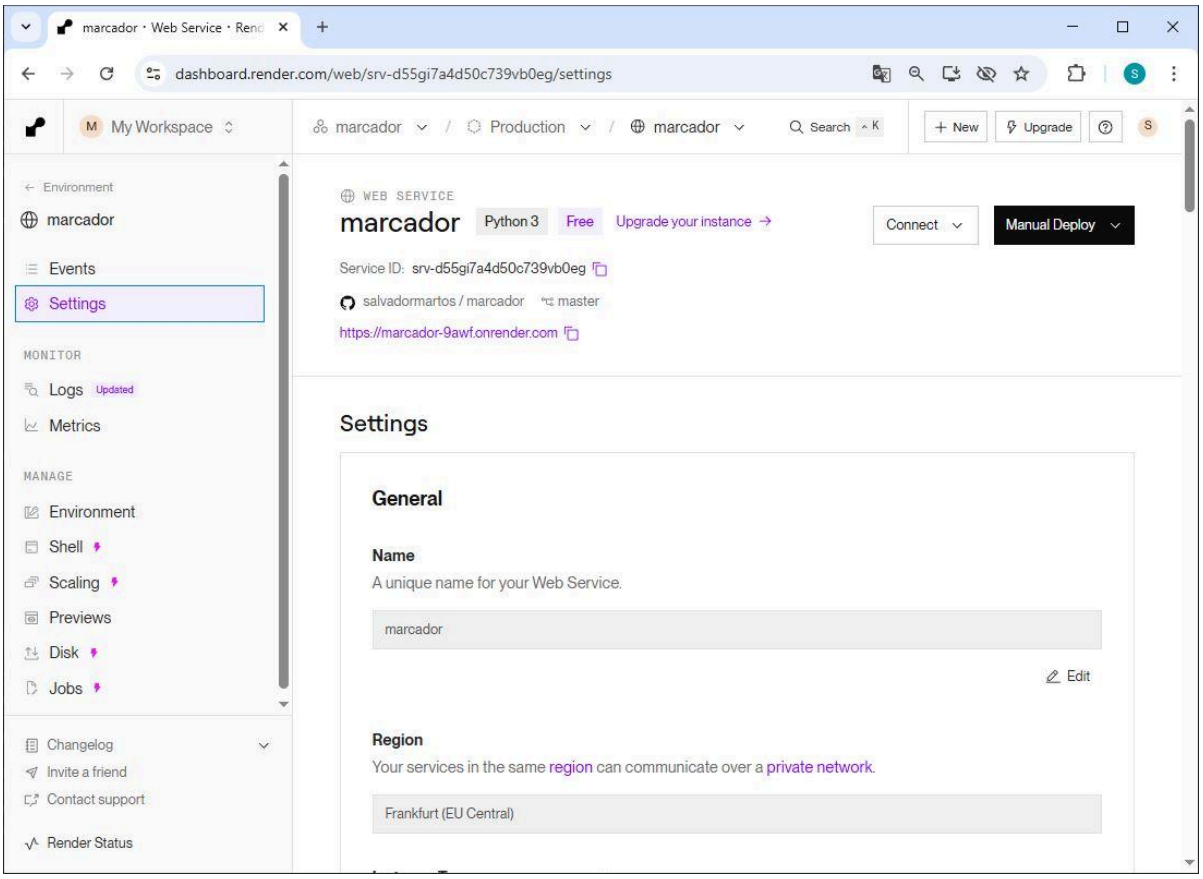
17 Django-marcador

```
# Static files (CSS, JavaScript, Images)
STATIC_URL = 'static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles') # Carpeta de destino para
producción
# Esto habilita la compresión y el cacheo de archivos (opcional pero
recomendado)
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

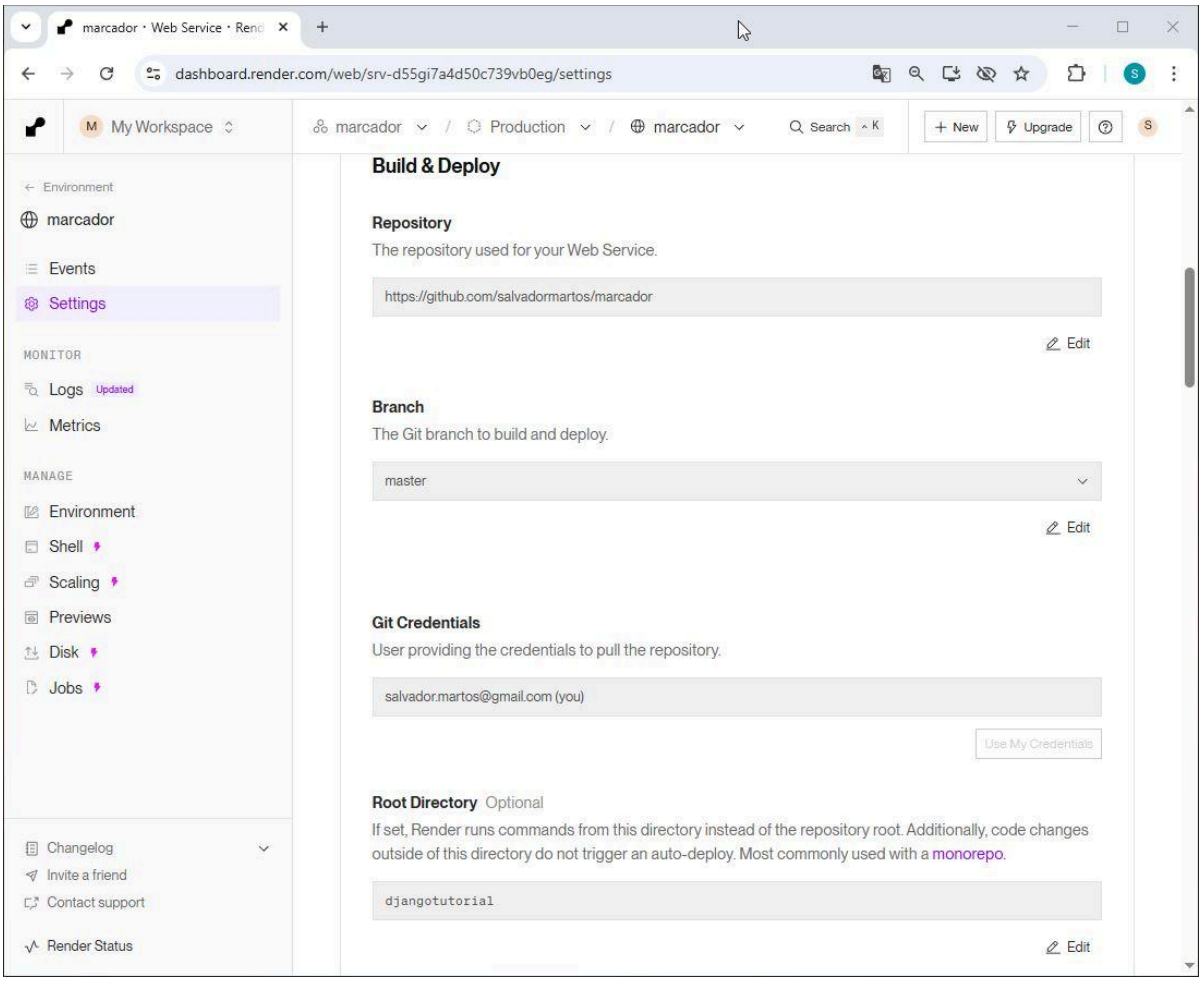
En Render quedaría así:



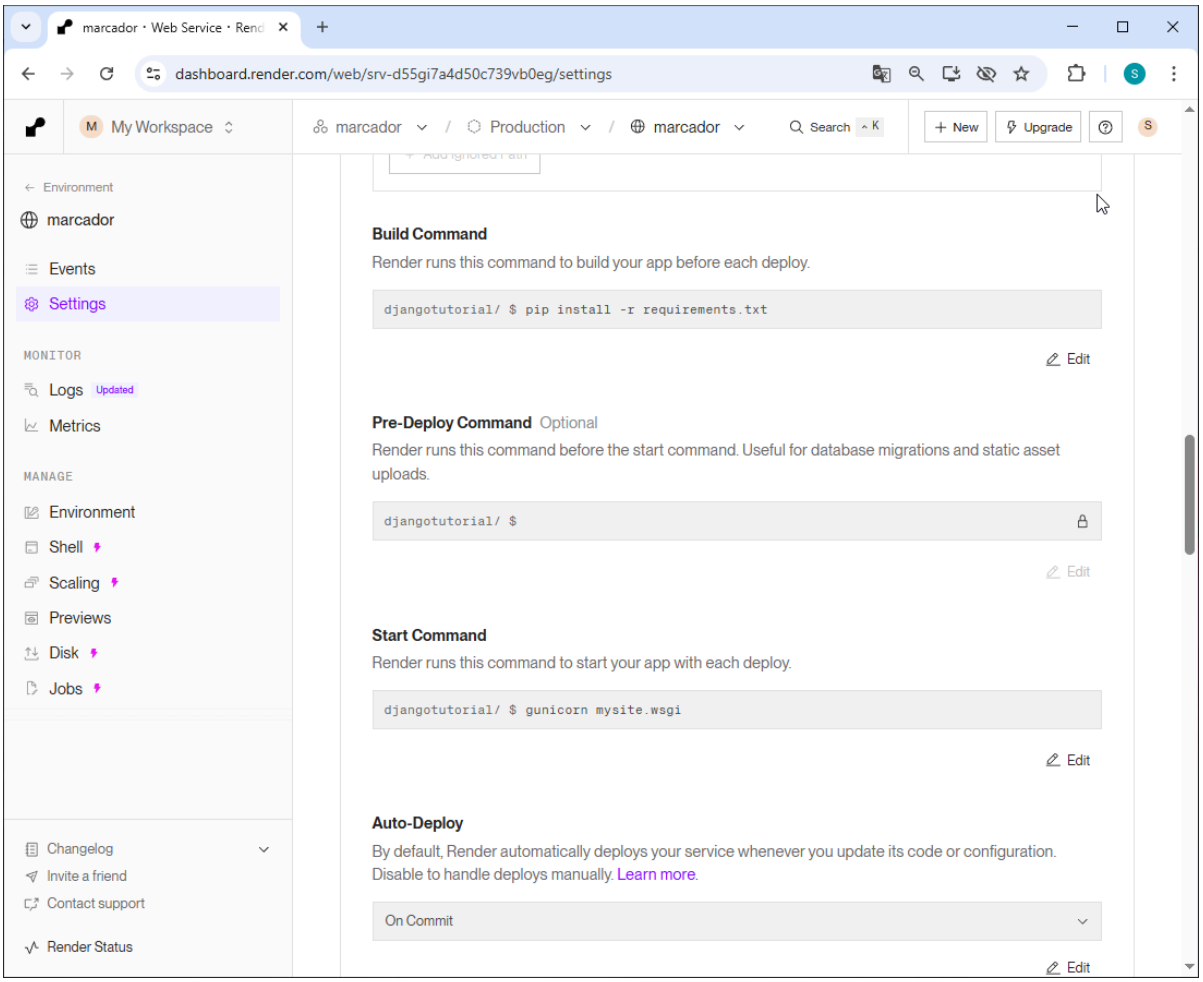
17 Django-marcador



17 Django-marcador



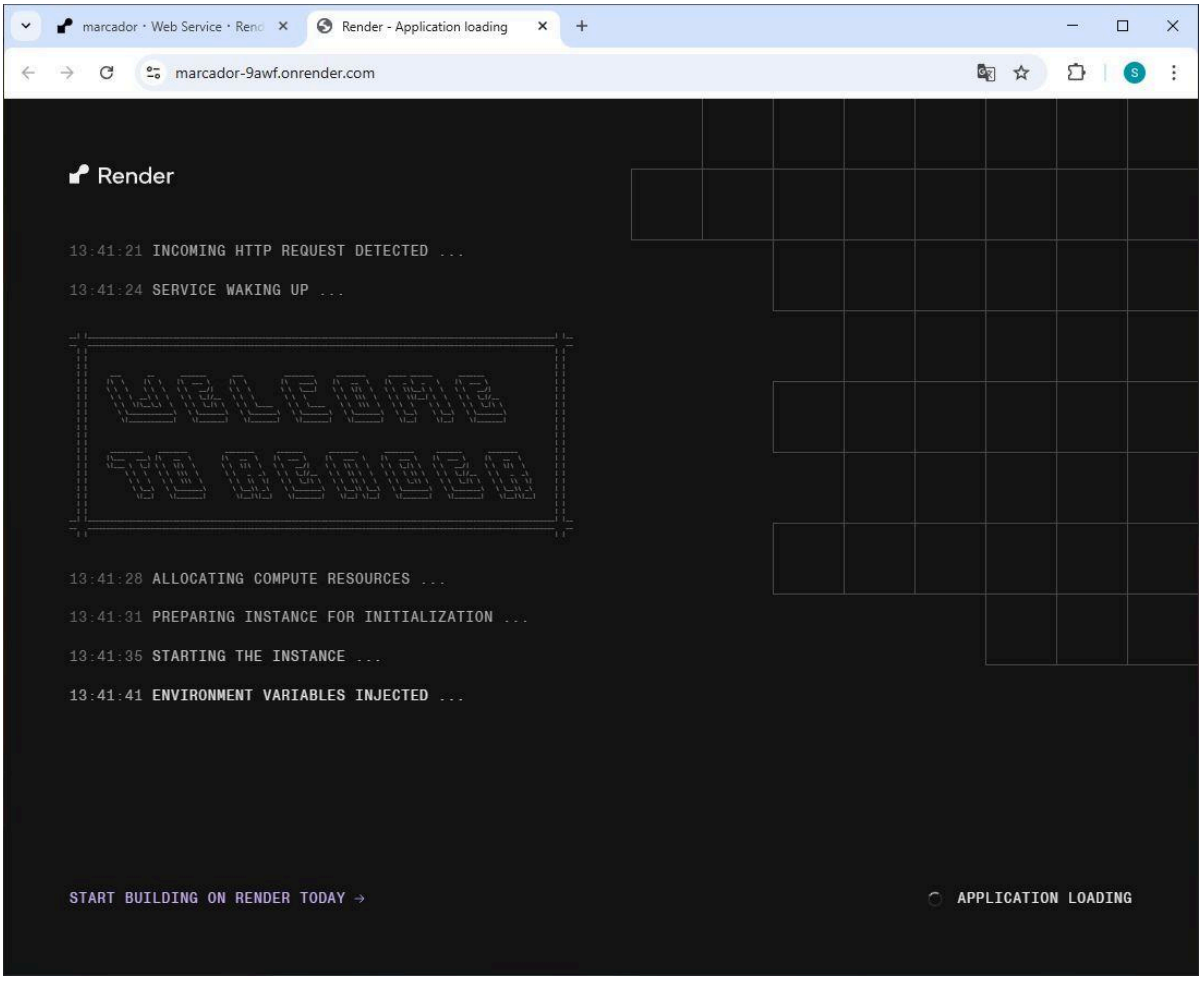
17 Django-marcador



Una vez hecho el despliegue podemos acceder a nuestra app: https://marcador-9awf.onrender.com/admin

Cuando hay inactividad la aplicación se echa a dormir, cuando vuelve la actividad tarda como un minuto en cargar, a caballo regalado … , mientras tanto muestra esta pantalla:

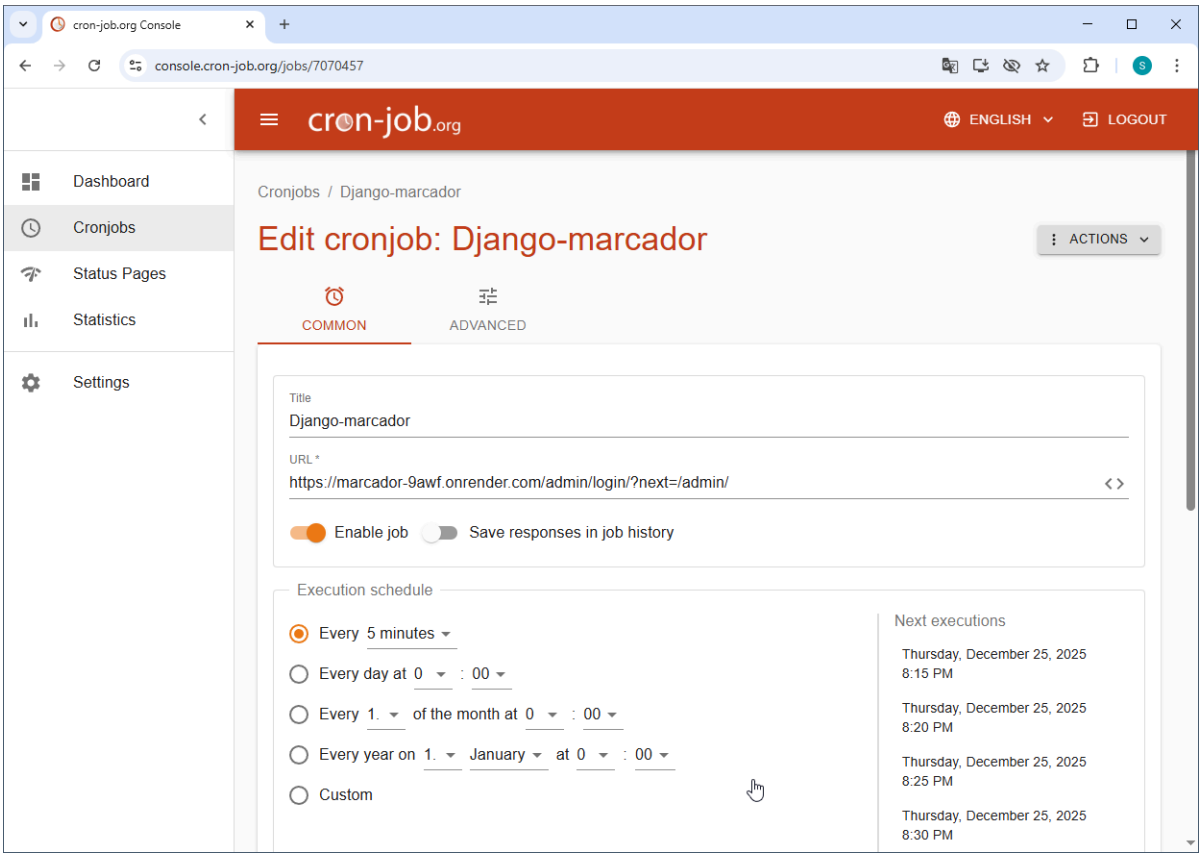
17 Django-marcador



Mantén tu app despierta

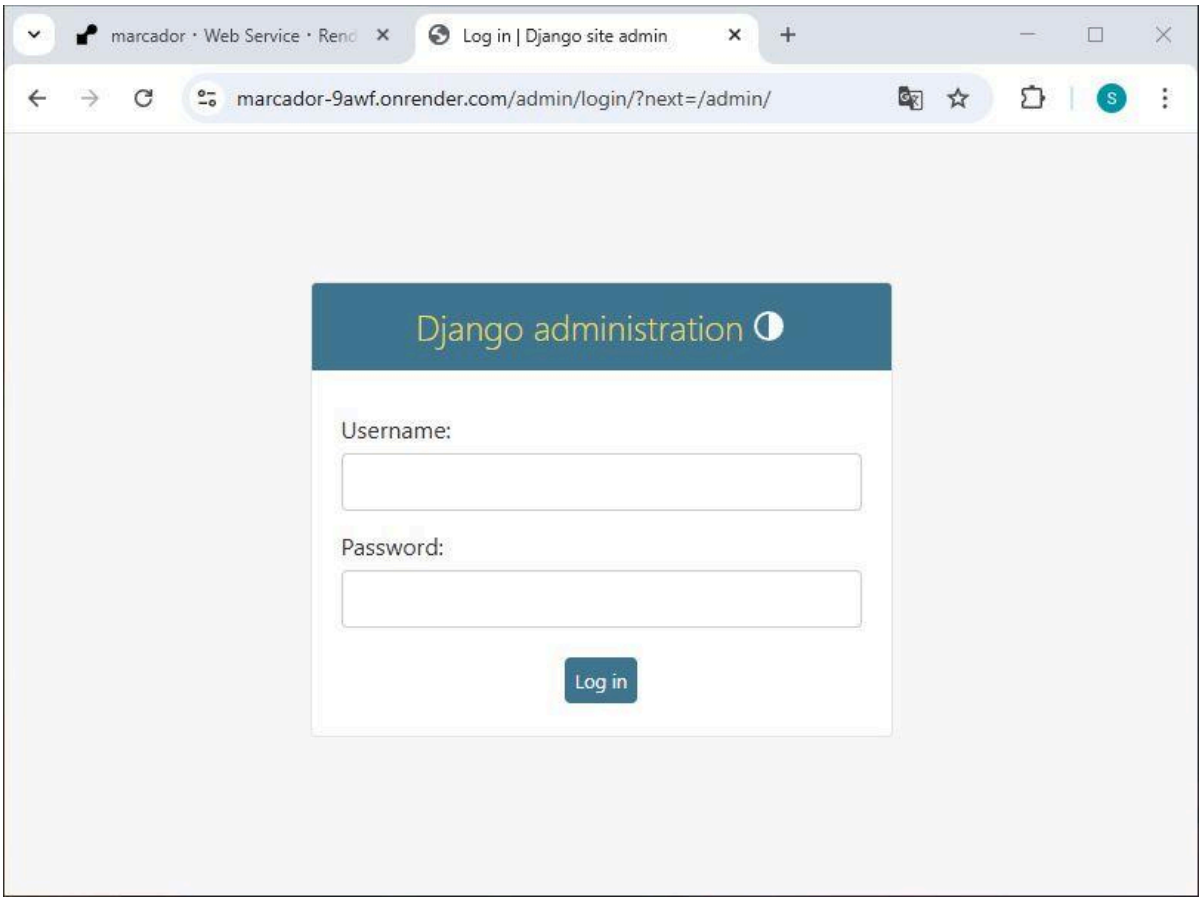
Si quieres evitar esto puedes hacer peticiones de tu app cada 5 minutos para mantenerla despierta, mediante cron-job.org,:

17 Django-marcador



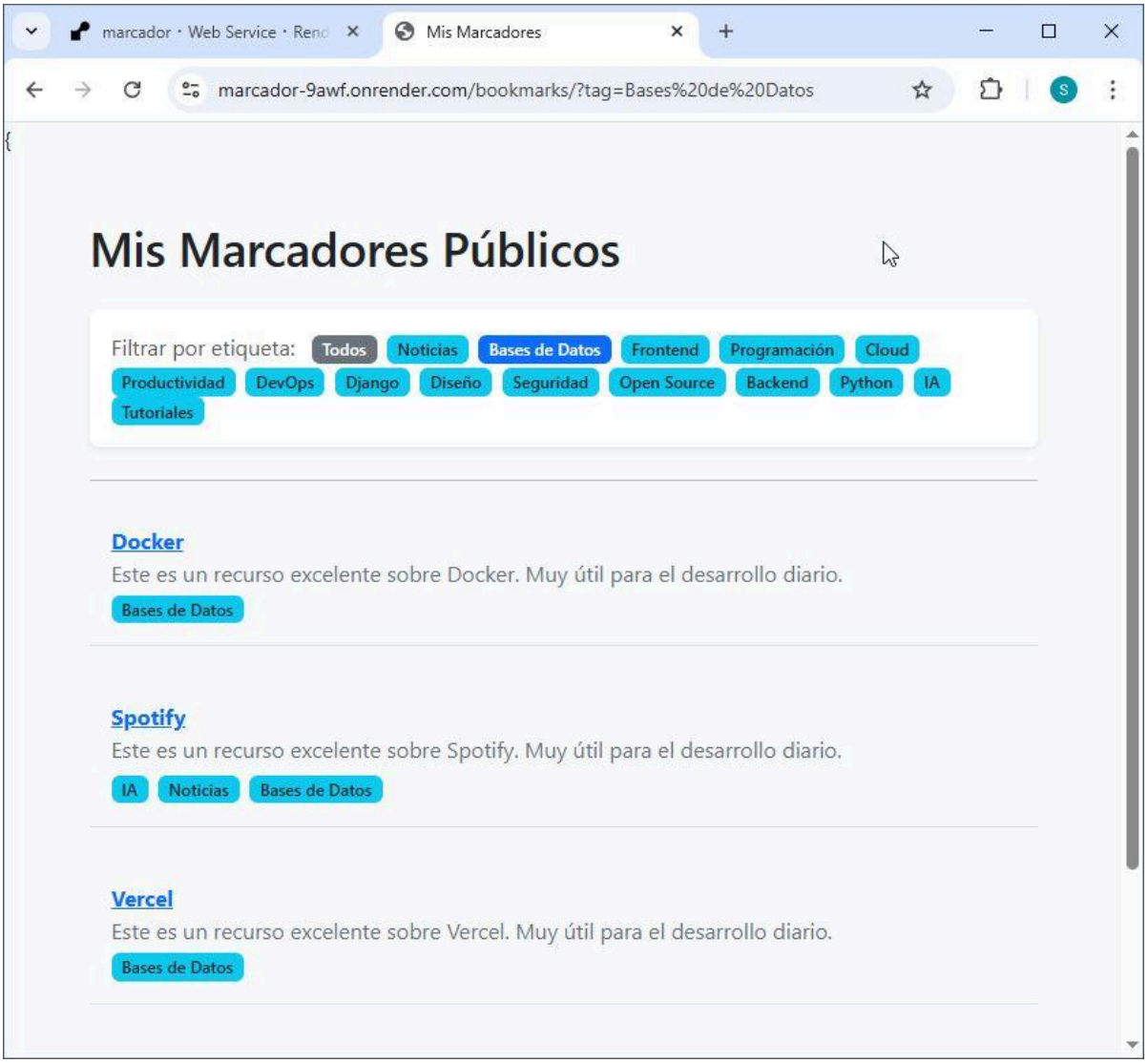
Nuestra app nos pide login: https://marcador-9awf.onrender.com/admin

17 Django-marcador



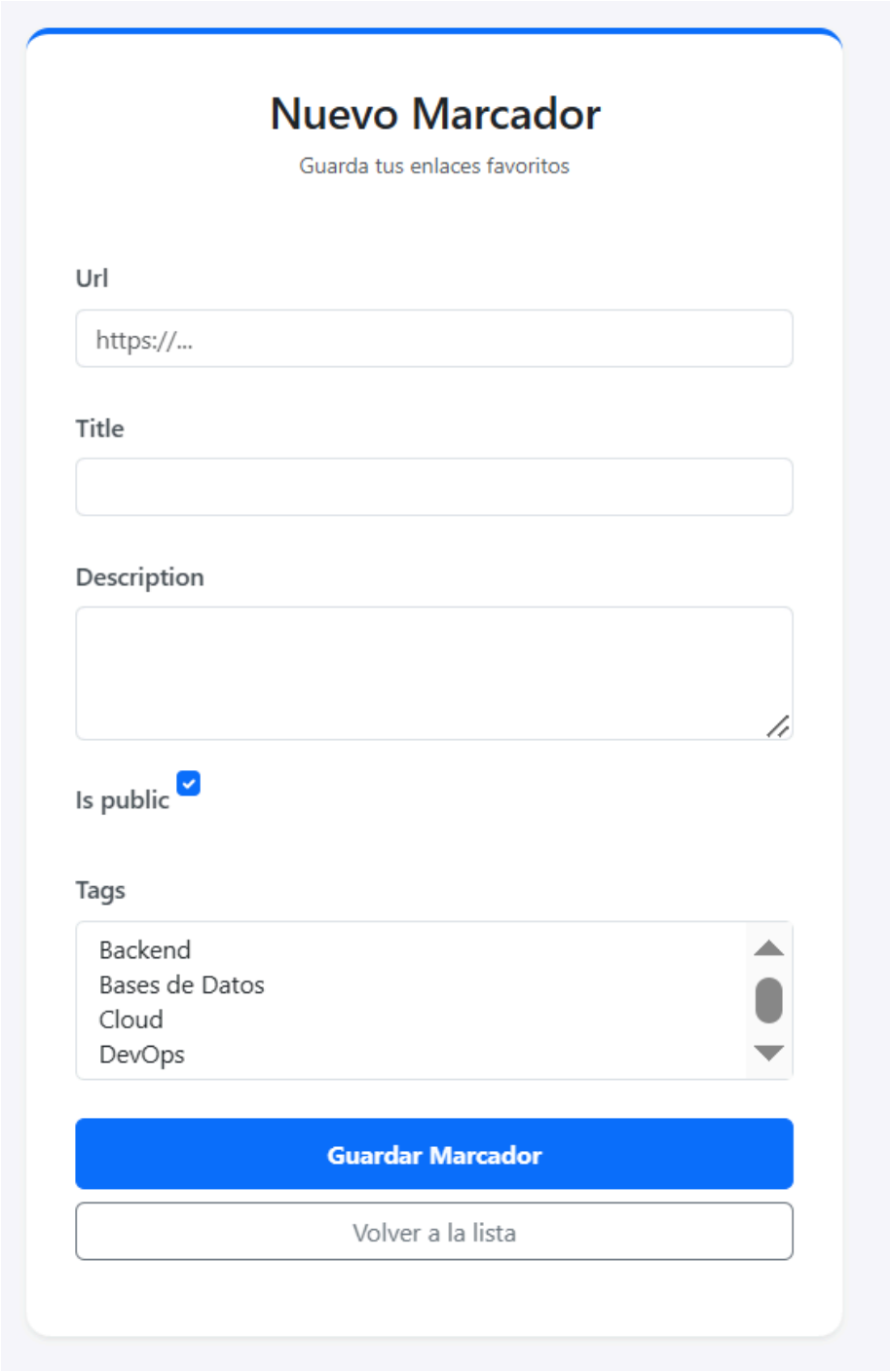
Podemos ver los marcadores del usuario:
https://marcador-9awf.onrender.com/bookmarks

17 Django-marcador



Podemos crear un nuevo marcador: https://marcador-9awf.onrender.com/nuevo/

17 Django-marcador



Puedes descargarte el código del proyecto en este enlace: [django-marcador](#).